AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

ZAAWANSOWANE PROGRAMOWANIE

Drzewo BST

Autor:
Wiktor Matusik
Mateusz Kowalczyk

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1. Ogólne określenie wymagań **3**

2. Analiza problemu **5**

2.1. Czym jest Drzewo BST 5

2.2. Operacje i sposoby przechodzenia przez drzewo 5

2.3. Zastosowania drzew BST 7

2.4. Sposób działania na przykładzie 7

2.5. Opis użytych narzędzi 8

3. Projektowanie **9**

3.1. Wykorzystywane narzędzia 9

3.2. Konfiguracja kompilatora 9

3.3. Sposób użycia Git oraz GitHub 10

4. Implementacja **11**

5. Wnioski **12**

Literatura **13**

Spis rysunków **13**

Spis tabel **14**

Spis listingów **15**

1. Ogólne określenie wymagań

Napisz program „drzewo BST" działającej w kontenerze STL w języku C++. Drzewo winno być zaimplementowana w klasie. Funkcjonalność (metod) drzewa:

- Dodaj element,
- Usuń element,
- Usuń całe drzewo,
- Szukaj drogi do podanego elementu,
- Wyświetl drzewo graficznie na ekranie, użytkownik wybiera metodę podczas wyświetlania (metody preorder, inorder, postorder) [oprogramuj wszystkie trzy],
- Zapis do pliku tekstowego wygenerowanego drzewa,

W drugiej klasie należy zaimplementować (metody) zapis do pliku i odczyt z pliku utworzonego drzewa BTS (plik musi być zapisany binarnie). Oprócz tego powinien mieć możliwość wczytania pliku tekstowego z cyframi, co daje to możliwość zbudowania drzewa. Program powinien posiadać możliwość wczytania pliku z liczbami do drzewa pustego lub istniejącego.

Funkcja main powinna wyświetlać menu z opcjami drzewa oraz odczytu i zapisu pliku. Program czeka na wybranie opcji. Nie zapomnij ustawić wyjścia z programu aby program można było poprawnie zamknąć.

Wszystkie utworzone klasy mają być zaimplementowane w oddzielnych plikach. Funkcja main także powinna być w osobnym pliku.

Celem zadania jest poznanie możliwości GitHuba w pracy nad projektem grupowym. Konto na GitHub jest utworzone. Należy utworzyć nowy projekt, który jest realizowany w dwuosobowych grupach.

Przy oddawaniu projektu należy zaprezentować:

- Co najmniej 5 commit'ów (każdej osoby),
- Najpierw jedna osoba tworzy gałąź i po kilku comitach ją scala. Po scaleniu druga osoba tworzy gałąź i po kilku comitach ją scala do głównej gałęzi ,
- Obie osoby w grupie mają utworzyć nowe gałęzie (w jednym punkcie obie osoby zaczynają pracę równoległą), a po kilku comitach wykonują scalenie do głównej gałęzi,

- Co najmniej 6 konfliktów, które należy rozwiązać (3 jedna osoba, 3 druga osoba) przy scalaniu gałęzi (w wcześniejszych punktach),

Przy pracy równoległej, a następnie scalaniu można używać wtyczki GitHub zainstalowanej do Visual Studio Code lub Visual Studio. Następnie sprawdźcie jak wygląda plik po scalaniu, który jest na GitHubie.

Dodatkowo do projektu należy dołączyć dokumentację w Latex z zastosowaniem uwag, które były omawiane na poprzednim projekcie. W dokumentacji wklej obraz drzewa z comitów , który należy pobrać z GitHuba oraz wygenerować dokumentacje Doxygenem. Projekt realizować bez użycia narzędzi sztucznej inteligencji takich jak ChatGPT, GitHub Copilot.

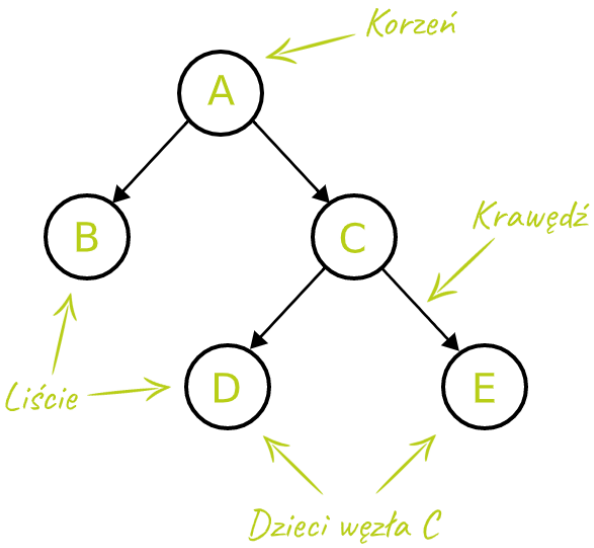
2. Analiza problemu

2.1. Czym jest Drzewo BST

Drzewo poszukiwań binarnych (ang. Binary Search Tree, BST) to rozgałęziona, hierarchiczna struktura danych, w której każdy węzeł przechowuje wartość oraz wskaźniki na co najwyżej dwóch potomków: lewego i prawego. Podstawową własnością drzewa BST jest porządek, w którym dla każdego węzła X:

- wszystkie wartości w lewym poddrzewie są mniejsze od wartości węzła X,
- wszystkie wartości w prawym poddrzewie są większe od wartości węzła X.

Dzięki tej właściwości średnia złożoność czasowa operacji wyszukiwania, dodawania oraz usuwania elementów wynosi O(log n), co czyni BST wydajną strukturą danych. Przykład drzewa BST na obrazku 2.1 (s. 5)



Rys. 2.1. Drzewo BST

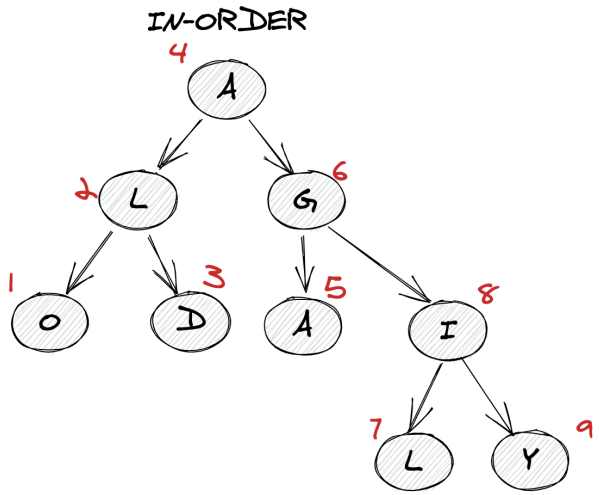
2.2. Operacje i sposoby przechodzenia przez drzewo

W drzewach BST istnieją trzy podstawowe metody przechodzenia i wyświetlania drzewa:

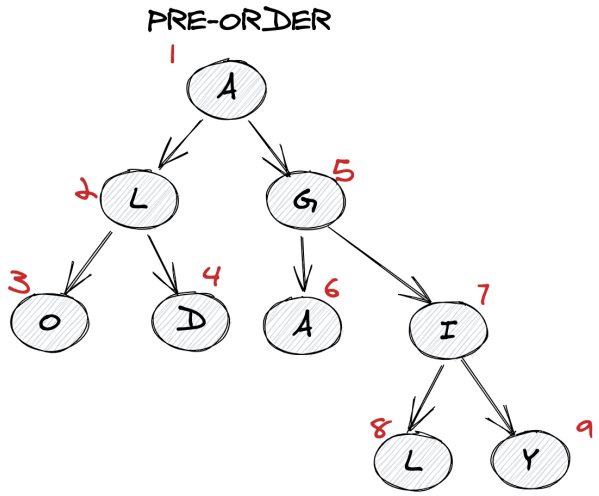
- **Inorder:** Odwiedza najpierw lewe poddrzewo, następnie węzeł główny, a na końcu prawe poddrzewo. Przeglądanie drzewa tą metodą zwraca elementy w kolejności rosnącej. Przykład na obrazku 2.2 (s. 6)

- **Preorder:** Odwiedza najpierw węzeł główny, potem lewe poddrzewo, a na końcu prawe poddrzewo. Używane m.in. do kopiowania drzewa lub zapisu jego struktury. Przykład na obrazku 2.3 (s. 6)

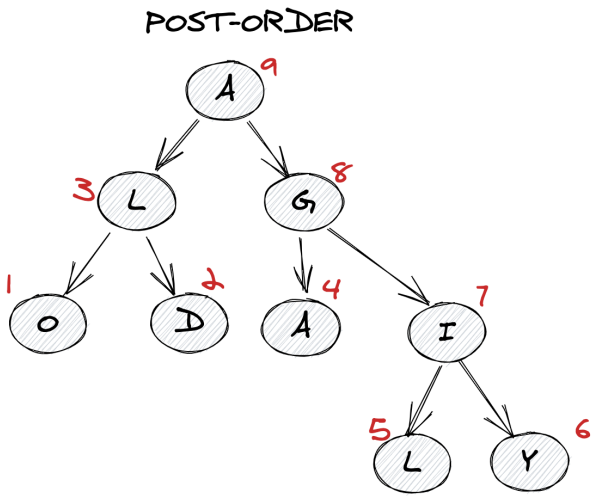
- **Postorder:** Odwiedza lewe poddrzewo, prawe poddrzewo, a na końcu węzeł główny. Metoda przydatna m.in. przy zwalnianiu pamięci. Przykład na obrazku 2.4 (s. 7)



Rys. 2.2. Drzewo BST Inorder



Rys. 2.3. Drzewo BST Preorder



Rys. 2.4. Drzewo BST Postorder

2.3. Zastosowania drzew BST

Drzewa poszukiwań binarnych znajdują szerokie zastosowanie w informatyce m.in. w:

- Systemach baz danych i indeksowaniu danych, gdzie wymagany jest szybki dostęp do posortowanych rekordów.
- Kompilatorach i parserach do reprezentacji wyrażeń arytmetycznych i hierarchii składniowej.
- Implementacji kontenerów asocjacyjnych w bibliotekach standardowych (np. `set` i `map` w C++ opartych na zbalansowanych odmianach BST).

2.4. Sposób działania na przykładzie

Proces dodawania elementów $50, 30, 70, 20$ do początkowo pustego drzewa BST:

- **Dodanie 50:** Drzewo jest puste, więc 50 staje się korzeniem (*root*).
- **Dodanie 30:** Wartość $30 < 50$, więc zostaje umieszczona jako lewy potomek węzła 50.
- **Dodanie 70:** Wartość $70 > 50$, więc trafia jako prawy potomek korzenia 50.
- **Dodanie 20:** Wartość $20 < 50$, przechodzimy w lewo do 30, następnie $20 < 30$, więc trafia jako lewy potomek węzła 30.

2.5. Opis użytych narzędzi

- **Git i GitHub:** Rozproszony system kontroli wersji oraz platforma do współdzielenia kodu, śledzenia historii zmian i synchronizacji pracy dwuosobowej grupy.
- **Visual Studio / Visual Studio Code:** Środowiska programistyczne wspomagające pisanie kodu w języku C++ oraz integrację z systemem Git za pomocą dedykowanych wtyczek.
- **Doxygen:** Narzędzie przeznaczone do automatycznej generacji dokumentacji na podstawie specjalnych komentarzy w kodzie źródłowym.

3. Projektowanie

3.1. Wykorzystywane narzędzia

Przy realizacji projektu drzewa BST wykorzystano następujące środowisko programistyczne oraz narzędzia pomocnicze:

- **Język programowania: C++** – język obiektowy umożliwiający bezpośrednie zarządzanie pamięcią dynamiczną oraz korzystanie z kontenerów biblioteki standardowej (STL).
- **Kompilator: MinGW-w64** – kompilator przeznaczony dla platformy Windows, odpowiedzialny za tłumaczenie kodu źródłowego C++ do pliku wykonywalnego .exe.
- **Środowisko programistyczne (IDE): Visual Studio** – zintegrowane środowisko programistyczne wyposażone w zaawansowany edytor kodu, menedżer projektów typu Solution, wbudowany debuger oraz natywne wsparcie dla systemu kontroli wersji Git.
- **System kontroli wersji i platforma programistyczna: Git i GitHub** – narzędzia służące do śledzenia historii zmian w kodzie, pracy współdzielonej w zespole dwuosobowym, tworzenia gałęzi oraz rozwiązywania konfliktów scalania.
- **Generator dokumentacji: Doxygen** – narzędzie automatyzujące proces generowania dokumentacji technicznej z komentarzy umieszczonych w plikach nagłówkowych i źródłowych.

3.2. Konfiguracja kompilatora

Projekt zarządzany jest w ramach pliku rozwiązania (.sln) programu Visual Studio. Ustawienia kompilacji i optymalizacji kodu definiowane są bezpośrednio w panelu właściwości projektu:

- **Standard C++:** Program wykorzystuje nowoczesny standard C++20, co pozwala korzystać z najnowszych funkcji języka.
- **Ostrzeżenia kompilatora:** Poziom ostrzeżeń ustawiono na Level 3 z włączoną pełną zgodnością ze standardem, co pomaga w wykrywaniu błędów podczas pisania kodu.

- **Typ aplikacji:** Projekt jest skompilowany jako aplikacja konsolowa systemu Windows zoptymalizowana pod kątem zarządzania pamięcią.
- **Biblioteki:** Kod opiera się wyłącznie na standardowej bibliotece C++, dlatego Visual Studio automatycznie dołącza wszystkie niezbędne składniki systemowe bez potrzeby instalowania dodatkowych bibliotek.

3.3. Sposób użycia Git oraz GitHub

Praca nad projektem odbywa się w zespole dwuosobowym przy wykorzystaniu zintegrowanego w programie Visual Studio panelu Team Explorer / Git Changes oraz platformy GitHub.

4. Implementacja

5. Wnioski

Spis rysunków

2.1. Drzewo BST 5
2.2. Drzewo BST Inorder 6
2.3. Drzewo BST Preorder 6
2.4. Drzewo BST Postorder 7

Spis tabel

Spis listingów